

AUDITORÍA ARQUITECTÓNICA — Legado Patrimonial WSS

Reporte de Evaluación Pre-Producción | Fase 5.5

Auditor: Claude (Opus 4.6) — Rol: Arquitecto Senior

Fecha: 24 de marzo de 2026

Alcance: Código fuente integral del proyecto hasta Fase 5.5

Objetivo: Evaluación profunda antes de conectar Supabase real

RESUMEN EJECUTIVO

El proyecto tiene una base arquitectónica sólida y un Design System visualmente coherente. El motor de reproducción (Zustand + PersistentPlayer) es la pieza más madura del sistema — bien pensada, defensiva y robusta. Las páginas estáticas de Fase 5.5 son limpias y consistentes.

Sin embargo, hay **3 vulnerabilidades críticas** que deben resolverse antes de conectar datos reales, **5 problemas de nivel medio** que afectan mantenibilidad y rendimiento, y varias mejoras recomendadas para escalar con confianza.

Calificación general: 7.2 / 10 — Sólido para desarrollo, no listo para producción sin las correcciones críticas.

ÁREA 1: ARQUITECTURA NEXT.JS 15

Fortalezas

Separación Server/Client bien ejecutada en Fase 5.5. Las páginas `/el-legado`, `/estudios` y `/alabanza` son Server Components puros — sin `"use client"`, con `export const metadata` estático, datos definidos fuera del componente. Esto es exactamente lo que Next.js 15 espera: el servidor renderiza HTML completo, sin JavaScript innecesario en el cliente.

`/archivo` es el mejor ejemplo de arquitectura híbrida. El Server Component `(archivo/page.tsx)` hace el fetch a Supabase con `await`, normaliza parámetros y pasa los datos limpios al Client Component `(ArchivoPageClient)`. Este patrón debería ser el modelo para todas las páginas con interactividad.

El layout raíz es minimalista y correcto. Monta el `<Navbar />`, el `{children}` y el `<PersistentPlayer />` sin lógica innecesaria. El player está fuera del flujo principal, como debe ser.

🚫 VULNERABILIDAD CRÍTICA #1: `<main>` anidados

Archivo: `src/app/layout.tsx`

Problema: El layout raíz envuelve `{children}` en `<main>`. Pero TODAS las páginas (`/elegido`, `/estudios`, `/alabanza`, `/conferencia/[id]`) también tienen su propio `<main>`. Esto produce `<main><main>...</main></main>` en el HTML renderizado.

Impacto: Viola la especificación HTML (solo debe haber un `<main>` landmark por página). Los lectores de pantalla se confunden al encontrar dos landmarks principales. Google Lighthouse penaliza esto en auditorías de accesibilidad.

Corrección:

- Opción A: Eliminar `<main>` del layout y dejar que cada página lo defina (más flexible).
- Opción B: Mantener `<main>` en el layout y cambiar las páginas a `<div>` o `<section>`.

Recomendación: Opción A — eliminar `<main>` del layout. Cada página ya lo tiene y esto les da control individual.

⚠️ Problema Medio: `page.tsx` (Home) es "use client" completo

Archivo: `src/app/page.tsx`

Problema: La página de inicio tiene "use client" al inicio, lo que convierte TODO el componente en Client Component. Esto significa: sin metadata estática exportable (SEO nulo en la página más importante del sitio), todo el HTML se genera en el cliente, y el bundle de JavaScript incluye todo el código de la página.

Además: La Home importa `useConferencias` (hook de fetch) y `usePlayerStore` desde `@/store` (barrel import). La propiedad que lee es `conferencia_activa`, pero el store real exporta `currentTrack`. Esto sugiere que la Home usa una versión anterior del store que no se actualizó.

Corrección: Separar en Server Component (fetch + metadata) y Client Islands (grid interactivo + player padding). Seguir el patrón de `/archivo`.

⚠️ Problema Medio: `conferencia/[id]/page.tsx` no exporta metadata

Archivo: `src/app/conferencia/[id]/page.tsx`

Problema: Es "use client" completo. No puede exportar metadata. Cuando alguien comparte un link a una conferencia específica en redes sociales, no habrá título ni descripción — solo el metadata genérico del layout raíz.

Para un portal documental donde cada conferencia tiene título, ponente y fecha, esto es una pérdida significativa de SEO y compartibilidad.

Corrección: Cuando conecten Supabase, esta página debe migrar a Server Component con `generateMetadata()` dinámico y extraer la interactividad (botón play, iframe) a Client Components separados.

ÁREA 2: MOTOR DE REPRODUCCIÓN (Zustand + PersistentPlayer)

Fortalezas

El `playerStore.ts` es la pieza más madura del proyecto. La ingeniería defensiva es notable: cada valor que entra al store pasa por funciones de sanitización (`sanitizeTrack`, `sanitizePosition`, `sanitizeIsoString`, `sanitizeHistory`). Los datos corruptos en `localStorage` no crashean la aplicación — se descartan silenciosamente. Esto es trabajo de producción real.

El adaptador multi-key para `localStorage` es elegante. En lugar de guardar todo el estado en una sola clave de `localStorage` (lo que haría Zustand por defecto), el store distribuye los datos en las 4 claves del Master Plan (`lp_current_track`, `lp_playback_position`, etc.). Esto facilita debugging y migración futura.

El ciclo de vida de persistencia es completo. Cubre `beforeunload`, `pagehide` y `visibilitychange`. El guard `__lpPlayerStorePersistenceBound__` previene duplicación de listeners. Esto maneja correctamente los tres escenarios: cerrar pestaña, cambiar de pestaña, y navegación en móvil.

El `PersistentPlayer` maneja hidratación SSR correctamente. El patrón `hashydrated` + render `null` hasta hidratación evita mismatches de SSR. El `useShallow` previene re-renders innecesarios.

El throttle de sincronización (cada 5 segundos) es la decisión correcta. `handleTimeUpdate` se dispara ~4 veces por segundo, pero solo sincroniza al store cada 5 segundos reales de reproducción. Esto evita escrituras excesivas a `localStorage`.

🚫 VULNERABILIDAD CRÍTICA #2: Listeners de persistencia duplicados

Archivos: `playerStore.ts` (líneas 860-876) y `PersistentPlayer.tsx` (líneas 1071-1100)

Problema: Ambos archivos registran listeners para `beforeunload`, `pagehide` y `visibilitychange`. El store lo hace a nivel de módulo con `registerPersistenceLifecycle()`. El componente lo hace en un `useEffect`. El resultado: cada evento de cierre ejecuta `flushPlaybackSnapshot` DOS VECES.

Impacto:

- Doble escritura a `localStorage` en cada cierre/ocultamiento de pestaña.

- En `visibilitychange`, dos llamadas secuenciales a `syncPlayback` pueden causar entradas duplicadas en el historial si la lógica de `hasRecordedHistoryForCurrentTrack` no filtra correctamente.
- No es un bug visible, pero es una ineficiencia que se acumula.

Corrección: Eliminar los listeners del `useEffect` en `PersistentPlayer.tsx` y dejar que el store maneje la persistencia de emergencia (cierre de pestaña). El componente solo debería encargarse de la sincronización durante reproducción activa.

⚠ **Problema Medio:** `PlayerTrack` es demasiado escueto

Archivo: `playerStore.ts`

Problema: El tipo `PlayerTrack` solo tiene 3 campos:

```
typescript

type PlayerTrack = {
  conferencia_id: string;
  titulo: string;
  url_audio: string;
};
```

No incluye `artist`, `coverImage` ni `duration`. El `PersistentPlayer` muestra título y estado, pero no puede mostrar artista ni imagen de portada porque esos datos no existen en el store.

Contexto: La página `/conferencia/[id]` tiene esos datos en su mock (`audio.artist`, `audio.coverImage`, `audio.duration`) pero `handlePlayAudio` no los pasa al store. Cuando conecten datos reales y el reproductor funcione con audio verdadero, se verá incompleto.

Corrección: Ampliar `PlayerTrack` y actualizar `handlePlayAudio`:

```
typescript

type PlayerTrack = {
  conferencia_id: string;
  titulo: string;
  url_audio: string;
  artista?: string | null;
  portada_url?: string | null;
  duracion_total?: number | null;
};
```

ÁREA 3: DESIGN SYSTEM Y SEMÁNTICA

Fortalezas

El dorado (#D4AF37) se usa de forma consistente en todas las páginas de Fase 5.5. Botones primarios, bordes, íconos y gradientes usan el mismo valor hex. La corrección del color antiguo (#c6a55a) en /conferencia/[id] se aplicó correctamente.

Los landmarks semánticos (aria-labelledby) están bien implementados en /estudios y /alabanza. Cada sección tiene un aria-labelledby vinculado al id de su <h2>, que es el patrón correcto.

El glassmorphism es consistente: backdrop-blur-2xl, bg-white/6, bordes border-white/10 — el mismo lenguaje visual en todas las páginas.

🔴 VULNERABILIDAD CRÍTICA #3: Tres dorados diferentes coexisten

Archivos: globals.css, todas las páginas

Problema: El Design System tiene actualmente TRES tonos de dorado:

Origen	Color	Donde se usa
globals.css variable	#e2b857 (--color-gold)	PersistentPlayer, clases CSS utilitarias
Páginas Tailwind	#D4AF37	Todas las páginas de Fase 5.5
Variante light CSS	#f0d080 (--color-gold-light)	Solo en globals.css, no se usa en páginas

El PersistentPlayer usa var(--color-gold) que resuelve a #e2b857. Las páginas usan #D4AF37 hardcodeado en clases de Tailwind. El usuario final ve dos dorados ligeramente diferentes: uno en el reproductor y otro en el resto de la interfaz.

Corrección: Actualizar globals.css para que --color-gold: #D4AF37 y progresivamente migrar las páginas a usar var(--color-gold) en lugar de hex hardcodeado. Esto centraliza el cambio en un solo lugar.

⚠ Problema Medio: CSS utilitario no utilizado

Archivo: globals.css

Problema: Se definieron las clases .glass, .glass-strong, .glass-card, .btn-gold, .btn-ghost, .media-badge, .text-gold, .bg-gradient-hero, .border-gradient-gold, 7 animaciones con sus clases (.animate-glow, .animate-shimmer, .animate-float, etc.) y estilos .stagger-children.

De todas estas, solo `.glass-card` y `.stagger-children` se usan en la página Home. Las páginas de Fase 5.5 no usan NINGUNA — todo es Tailwind inline.

Impacto: CSS muerto que aumenta el bundle y crea confusión sobre cuál es el patrón oficial (¿clases utilitarias propias o Tailwind inline?).

Corrección: Decidir un camino. Si el proyecto usa Tailwind como estándar (que es lo que indican las páginas de Fase 5.5), mover estas clases a un archivo `legacy.css` o eliminarlas. Si quieren mantener el sistema propio, migrar las páginas a usarlo.

Problema Menor: Fondos inconsistentes

Las páginas usan tres fondos oscuros diferentes: `globals.css` define `--color-bg-primary: #0a0a14`, `/el-legado` y `/estudios` usan `bg-[#050505]`, `/conferencia/[id]` usa `bg-[#0a0a0a]`. Son variaciones sutiles pero técnicamente inconsistentes.

ÁREA 4: CONTRATO DE DATOS (Preparación para Supabase)

Fortalezas

La estructura de tipos en las páginas mock es limpia y escalable. Los tipos como `StudyCollection`, `MusicCollection`, `FeaturedTrack` reflejan bien las entidades que existirán en Supabase. Los slugs están preparados para rutas dinámicas. Los campos opcionales usan `| null` correctamente.

El patrón de datos fuera del componente es correcto. Todas las constantes (`STUDY_COLLECTIONS`, `MUSIC_COLLECTIONS`, etc.) están definidas antes del componente, no dentro de él. Esto evita recrearlos en cada render y facilita la migración a fetches de Supabase.

`/archivo` ya está conectado a Supabase con un patrón sólido. `getArchivoCronologicoPage` recibe filtros y paginación, devuelve `{ data, total }`. Este contrato es el modelo a seguir.

⚠ Problema Medio: Discrepancia de nomenclatura entre store y página

Archivos: `playerStore.ts` vs `conferencia/[id]/page.tsx`

Problema: El store usa nomenclatura en español (`conferencia_id`, `titulo`, `url_audio`). El mock de la conferencia usa inglés (`id`, `title`, `url`, `artist`, `duration`). `handlePlayAudio` traduce entre ambos:

```
typescript
```

```
playTrack({
  conferencia_id: conference.id,
  titulo: conference.audio.title,
  url_audio: conference.audio.url,
});
```

Cuando conecten Supabase, la tabla probablemente use español (siguiendo el patrón del store). Pero los tipos del mock de conferencia usan inglés. Esto creará una capa de traducción innecesaria en cada componente que llame a `playTrack`.

Corrección: Definir una convención única. Dado que el store y el Master Plan usan español, los tipos de datos de Supabase deberían seguir español. Los contratos provisionales deberían adaptarse desde ahora.

⚠ Problema Medio: Home usa propiedades que no existen en el store

Archivo: `src/app/page.tsx`

Problema: La Home lee `conferencia_activa` del store, pero `playerStore.ts` exporta `currentTrack`. También importa desde `@/store` (barrel) mientras que la conferencia importa desde `@/store/playerStore`. Esto indica que la Home se quedó en una versión anterior del store y no se actualizó junto con la Fase 5.4.

Corrección: Actualizar la Home para usar `currentTrack` y los imports directos al archivo del store.

ÁREA 5: DEUDA TÉCNICA Y ESCALABILIDAD

Deuda identificada (ordenada por impacto)

1. No existen archivos `loading.tsx`, `error.tsx` ni `not-found.tsx` a nivel de app.

Cuando el usuario navegue a una ruta inexistente o un fetch de Supabase falle, Next.js mostrará su página de error por defecto (blanca, genérica, en inglés). Para un portal con Design System premium, esto es inaceptable. Crear estos archivos con el estilo del proyecto.

2. No hay estrategia de caché definida.

Con 16,000+ archivos y miles de conferencias, las consultas a Supabase necesitan una estrategia de caché. Next.js 15 con App Router ofrece `revalidate` estático e ISR. El Master Plan no menciona caché. Sin esto, cada visita a una conferencia popular genera una consulta a Supabase.

3. El `PersistentPlayer` no tiene UI de error de carga.

Si el audio falla (URL rota, timeout, formato incompatible), el player solo captura el error de `play()` y pone `isPlaying: false`. No muestra al usuario que algo falló. Con 5,000 audios de diferentes épocas y calidades, las fallas de carga serán frecuentes.

4. No hay un componente `PlayButton` reutilizable.

Actualmente, la lógica de reproducción vive inline en `handlePlayAudio` de la conferencia. Las páginas `/alabanza` tienen botones desactivados esperando conexión futura. Cuando llegue el momento, cada página va a reimplementar la misma lógica. Un componente `PlayButton` con `"use client"` que encapsule la conexión al store sería la solución.

5. Las colecciones de estudios y alabanza no comparten tipos.

`StudyCollection` y `MusicCollection` son tipos casi idénticos pero definidos por separado. Cuando Supabase entre, probablemente sean la misma tabla con un campo `type`. Unificar los tipos ahora facilita la migración.

CHECKLIST DE CORRECCIONES PRIORITARIAS

Antes de conectar Supabase (bloqueo)

- ☐ Eliminar `<main>` duplicado (layout vs páginas)
- ☐ Unificar `--color-gold` en `globals.css` a `#D4AF37`
- ☐ Corregir la Home: actualizar `conferencia_activa` → `currentTrack`, añadir metadata
- ☐ Eliminar listeners duplicados de persistencia en `PersistentPlayer.tsx`

Antes de primera release (importante)

- ☐ Crear `loading.tsx`, `error.tsx` y `not-found.tsx` con Design System
- ☐ Ampliar `PlayerTrack` con `artista`, `portada_url`, `duracion_total`
- ☐ Migrar `/conferencia/[id]` a Server Component + Client Islands
- ☐ Crear componente `PlayButton.tsx` reutilizable
- ☐ Limpiar CSS no utilizado en `globals.css`
- ☐ Unificar fondos oscuros en una sola variable

Mejoras recomendadas (escalabilidad)

- ☐ Definir estrategia de caché (ISR / revalidate) para páginas de conferencia
 - ☐ Agregar UI de error en `PersistentPlayer` cuando el audio falla
 - ☐ Unificar tipos de colección (estudios + alabanza)
 - ☐ Estandarizar nomenclatura (español en toda la capa de datos)
-

CONCLUSIÓN

El proyecto tiene buen ADN arquitectónico. Las decisiones fundamentales (Next.js 15 App Router, Zustand con persistencia defensiva, separación Server/Client, arquitectura híbrida de

storage) son las correctas. El Design System es visualmente coherente y las páginas de Fase 5.5 muestran que el equipo internaliza las correcciones iterativamente.

Las vulnerabilidades críticas son todas corregibles en unas pocas horas de trabajo. No hay errores de diseño que requieran reescribir componentes enteros. La deuda técnica es manejable si se atiende antes de la conexión a datos reales.

Recomendación final: Corregir las 4 tareas marcadas como "bloqueo", crear los boundary files (`loading`), (`error`), (`not-found`), y luego proceder con (`/podcast`) y (`/blog`) con confianza.

Auditoría generada por Claude (Opus 4.6) para el proyecto Legado Patrimonial WSS.